

Static and This Keyword in Java

A. Java static Keyword

The static keyword in Java is used to declare members that belong to the class itself rather than to individual objects. Normally, when a class is instantiated, each object receives its own copy of instance variables and methods. When a member is declared static, there is generally one class-level member shared by all objects of that class.

General Syntax

```
static dataType variableName;  
  
static returnType methodName() {  
    // statements  
}
```

Example

```
class Student {  
    String name;  
    static String school = "Cavite State University";  
}
```

Here, name is an instance variable while school is a static variable. Each Student object has its own name, while all Student objects share the same school.

Creating Objects

```
public class Main {  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
        Student s2 = new Student();  
  
        s1.name = "Juan";  
        s2.name = "Maria";  
  
        System.out.println(s1.name);  
        System.out.println(s2.name);  
  
        System.out.println(Student.school);  
    }  
}
```

Output:

```
Juan  
Maria  
Cavite State University
```

B. Java Static Variable

A static variable is a variable that belongs to the class rather than to a particular object. It is also commonly called a class variable or static field. A static variable is useful when a value should be shared among all objects.

Example 1: Shared School Name

```
class Student {
    String name;
    static String school = "CvSU";
}

public class Main {
    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student();
        Student s3 = new Student();

        s1.name = "Juan";
        s2.name = "Maria";
        s3.name = "Pedro";

        System.out.println(s1.name + " - " + Student.school);
        System.out.println(s2.name + " - " + Student.school);
        System.out.println(s3.name + " - " + Student.school);
    }
}
```

Output:

```
Juan - CvSU
Maria - CvSU
Pedro - CvSU
```

Example 2: Counting Objects

```
class Student {

    String name;
    static int studentCount = 0;

    Student(String name) {
        this.name = name;
        studentCount++;
    }
}

public class Main {
    public static void main(String[] args) {

        Student s1 = new Student("Juan");
        Student s2 = new Student("Maria");
        Student s3 = new Student("Pedro");

        System.out.println("Number of students: "
            + Student.studentCount);
    }
}
```

DCIT 50 – Object Oriented Programming

```
}  
}
```

Output:

```
Number of students: 3
```

There is only one shared copy of studentCount. Every time a Student object is created, studentCount is incremented.

Instance Variable vs Static Variable

Instance Variable	Static Variable
Belongs to an object	Belongs to a class
Each object has its own copy	Shared by all objects
Created for each object	One shared class-level copy
Accessed through an object	Preferably accessed through class name
Example: name	Example: studentCount

C. Java Static Method

A static method is a method that belongs to the class rather than an individual object. It can be called without creating an object.

Syntax

```
static returnType methodName() {  
    // statements  
}
```

Example

```
class Calculator {  
  
    static int add(int a, int b) {  
        return a + b;  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        int result = Calculator.add(10, 20);  
  
        System.out.println("Result: " + result);  
    }  
}
```

Output:

```
Result: 30
```

Another Example

```
class MathUtility {  
  
    static int square(int number) {  
        return number * number;  
    }  
}
```

DCIT 50 – Object Oriented Programming

```
        static int cube(int number) {
            return number * number * number;
        }
    }

    public class Main {
        public static void main(String[] args) {

            System.out.println(MathUtility.square(5));
            System.out.println(MathUtility.cube(3));
        }
    }
}
```

Output:

```
25
27
```

Why is main() Static?

The standard Java entry point is public static void main(String[] args). The main method is static so that the JVM can execute it without first creating an object of the class.

```
public class Main {

    public static void main(String[] args) {
        System.out.println("Program started.");
    }
}
```

D. Restrictions for Static Methods

Restriction 1: A static method cannot directly access an instance variable

```
class Student {

    String name;

    static void displayName() {
        System.out.println(name); // ERROR
    }
}
```

The static method does not know which object's name should be used.

Correct Version

```
class Student {

    String name;

    static void displayName(Student student) {
        System.out.println(student.name);
    }
}
```

DCIT 50 – Object Oriented Programming

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Student s = new Student();  
        s.name = "Juan";  
  
        Student.displayName(s);  
    }  
}
```

Output:

```
Juan
```

Restriction 2: A static method cannot directly access an instance method

```
class Student {  
  
    void display() {  
        System.out.println("Student");  
    }  
  
    static void show() {  
        display(); // ERROR  
    }  
}
```

Correct approach: create or receive an object and call the instance method through that object.

```
class Student {  
  
    void display() {  
        System.out.println("Student");  
    }  
  
    static void show() {  
        Student s = new Student();  
        s.display();  
    }  
}
```

Restriction 3: this cannot be used in a static context

```
class Student {  
  
    String name;  
  
    static void display() {  
        System.out.println(this.name); // ERROR  
    }  
}
```

The this keyword refers to the current object. A static method belongs to the class and does not have a current object.

DCIT 50 – Object Oriented Programming

What Can a Static Method Access?

- Static variables
- Other static methods

```
class Example {  
  
    static int number = 10;  
  
    static void method1() {  
        System.out.println("Method 1");  
    }  
  
    static void method2() {  
        System.out.println(number);  
        method1();  
    }  
}
```

E. Java Static Block

A static block is a block of code declared using the static keyword. It is executed when the class is initialized by the JVM, before normal use of the class proceeds.

Syntax

```
static {  
    // statements  
}
```

Example

```
class Student {  
  
    static String school;  
  
    static {  
        school = "Cavite State University";  
        System.out.println("Static block executed.");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println("Main method started.");  
  
        System.out.println(Student.school);  
    }  
}
```

Output:

```
Static block executed.  
Main method started.  
Cavite State University
```

DCIT 50 – Object Oriented Programming

Static Block for Initialization

```
class University {  
  
    static String name;  
    static int establishedYear;  
  
    static {  
        name = "Cavite State University";  
        establishedYear = 1906;  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println(University.name);  
        System.out.println(University.establishedYear);  
    }  
}
```

Output:

```
Cavite State University  
1906
```

Multiple Static Blocks

```
class Example {  
  
    static {  
        System.out.println("Static Block 1");  
    }  
  
    static {  
        System.out.println("Static Block 2");  
    }  
  
    static {  
        System.out.println("Static Block 3");  
    }  
}
```

They execute from top to bottom when the class is initialized.

```
Static Block 1  
Static Block 2  
Static Block 3
```

F. Java this Keyword

The this keyword refers to the current object. If s1.display() is called, then inside display(), this refers to s1.

```
class Student {
```

DCIT 50 – Object Oriented Programming

```
String name;

void display() {
    System.out.println(this.name);
}

public class Main {

    public static void main(String[] args) {

        Student student = new Student();

        student.name = "Juan";
        student.display();
    }
}
```

Output:

```
Juan
```

G. Different Ways to Use the this Keyword

1. this to Refer to the Current Object

```
class Student {

    String name;

    void display() {
        System.out.println(this.name);
    }
}
```

2. this to Resolve Variable Shadowing

One of the most common uses of this is distinguishing an instance variable from a parameter having the same name.

```
class Student {

    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + this.name);
        System.out.println("Age: " + this.age);
    }
}
```


DCIT 50 – Object Oriented Programming

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student("Juan", 20);  
        Student s2 = new Student("Maria", 21);  
  
        s1.display();  
        s2.display();  
    }  
}
```

Output:

```
Name: Juan  
Age: 20  
Name: Maria  
Age: 21
```

3. this to Call the Current Class Method

```
class Student {  
  
    void study() {  
        System.out.println("Studying Java...");  
    }  
  
    void start() {  
        System.out.println("Student started.");  
        this.study();  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Student student = new Student();  
        student.start();  
    }  
}
```

Output:

```
Student started.  
Studying Java...
```

4. this to Call Another Constructor

The this() syntax can call another constructor within the same class. This is called constructor chaining.

```
class Student {  
  
    String name;  
    int age;  
  
    Student() {
```

DCIT 50 – Object Oriented Programming

```
        this("Unknown", 0);
    }

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

public class Main {

    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student("Juan", 20);

        s1.display();
        s2.display();
    }
}
```

Output:

```
Name: Unknown
Age: 0
Name: Juan
Age: 20
```

Important rule: When this() is used to call another constructor, it must be the first statement in the constructor.

5. this to Pass the Current Object as an Argument

```
class Student {

    String name;

    Student(String name) {
        this.name = name;
    }

    void register(Student student) {
        System.out.println(student.name + " is registered.");
    }

    void enroll() {
        register(this);
    }
}

public class Main {
```

DCIT 50 – Object Oriented Programming

```
public static void main(String[] args) {  
  
    Student student = new Student("Juan");  
    student.enroll();  
}  
}
```

Output:

```
Juan is registered.
```

Here, register(this) passes the current Student object as an argument.

6. this to Return the Current Object

A method can return the current object using this. This is commonly used for method chaining.

```
class Student {  
  
    String name;  
    int age;  
  
    Student setName(String name) {  
        this.name = name;  
        return this;  
    }  
  
    Student setAge(int age) {  
        this.age = age;  
        return this;  
    }  
  
    void display() {  
        System.out.println(name);  
        System.out.println(age);  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Student student = new Student();  
  
        student.setName("Juan")  
            .setAge(20)  
            .display();  
    }  
}
```

Output:

```
Juan  
20
```